



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Zadanie: Z3

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Operatory do obsługi łańcuchów

W języku C# łańcuchy znaków (string) są obiektami klasy String. Klasa ta oferuje wiele metod i operatorów do manipulacji łańcuchami. Poniżej omówimy najważniejsze operatory i metody:

Operator konkatencji (+):

Służy do łączenia dwóch lub więcej łańcuchów.

```
string firstName = "John";  
string lastName = "Rambo";  
string fullName = firstName + " " + lastName; // "John Rambo"
```

Operator porównania (==, !=):

Umożliwia porównywanie dwóch łańcuchów.

```
string str1 = "hello";  
string str2 = "world";  
bool areEqual = str1 == str2; // false
```

Metoda String.Concat:

Alternatywa dla operatora + do łączenia łańcuchów.

```
string firstName = "John";  
string lastName = "Rambo";  
string fullName = String.Concat(firstName, " ", lastName); // "John Rambo"
```

Metoda String.Compare:

Służy do porównywania dwóch łańcuchów z możliwością określenia reguł porównania.

```
int result = String.Compare(str1, str2); // wynik < 0, ponieważ "hello" jest mniejsze niż "world"
```

Interpolacja łańcuchów (\$):

Umożliwia wstawianie wartości zmiennych bezpośrednio w łańcuchu.

```
string firstName = "John";  
string lastName = "Rambo";  
string fullName = $"{firstName} {lastName}"; // "John Rambo"
```

Metoda String.Format:

Umożliwia formatowanie łańcuchów z użyciem indeksów.

```
string firstName = "John";  
string lastName = "Rambo";  
string fullName = String.Format("{0} {1}", firstName, lastName); // "John Rambo"
```

Interpolacja łańcuchów:

```
string name = "Alicja";  
int age = 30;  
string info = $"Name: {name}, Age: {age}"; // "Name: Alicja, Age: 30"
```

Instrukcja warunkowa i instrukcja wyboru

Instrukcje warunkowe i instrukcje wyboru są podstawowymi elementami sterowania przepływem programu w języku C#. Pozwalają one na podejmowanie decyzji w kodzie na podstawie określonych warunków.

Instrukcja if: Umożliwia wykonanie bloku kodu, jeśli warunek jest spełniony.

```
int a = 10;  
if (a > 5)  
{  
    Console.WriteLine("a jest większe niż 5");  
}
```

Instrukcja if-else: Umożliwia wykonanie jednego z dwóch bloków kodu w zależności od tego, czy warunek jest spełniony.

```
int a = 10;  
if (a > 5)  
{  
    Console.WriteLine("a jest większe niż 5");  
}  
else  
{  
    Console.WriteLine("a nie jest większe niż 5");  
}
```

Instrukcja else if: Umożliwia sprawdzenie wielu warunków po kolei.

```
int a = 10;  
if (a > 10)  
{  
    Console.WriteLine("a jest większe niż 10");  
}  
else if (a == 10)  
{  
    Console.WriteLine("a jest równe 10");  
}
```

```
else
{
    Console.WriteLine("a jest mniejsze niż 10");
}
```

Instrukcja switch: Umożliwia wybór jednego z wielu bloków kodu na podstawie wartości zmiennej.

```
int day = 3;
switch (day)
{
    case 1:
        Console.WriteLine("Poniedziałek");
        break;
    case 2:
        Console.WriteLine("Wtorek");
        break;
    case 3:
        Console.WriteLine("Środa");
        break;
    default:
        Console.WriteLine("Nieznany dzień");
        break;
}
```

Instrukcje pętli

Instrukcje pętli są fundamentalnym elementem programowania, pozwalającym na wielokrotne wykonywanie określonego bloku kodu. W języku C# mamy kilka rodzajów pętli, które różnią się sposobem działania i zastosowaniem.

Pętla for: Używana, gdy znana jest liczba iteracji. Składa się z trzech części: inicjalizacji, warunku i inkrementacji/dekrementacji.

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

Pętla while: Używana, gdy liczba iteracji nie jest z góry znana. Warunek jest sprawdzany przed każdą iteracją.

```
int i = 0;
while (i < 10)
{
    Console.WriteLine(i);
    i++;
}
```

Pętla do-while: Podobna do pętli while, ale warunek jest sprawdzany po wykonaniu bloku kodu. Gwarantuje przynajmniej jedno wykonanie pętli.

```
int i = 0;
do
{
    Console.WriteLine(i);
    i++;
} while (i < 10);
```


Pętla foreach: Używana do iteracji po elementach kolekcji (np. tablicy, listy). Nie wymaga ręcznego zarządzania indeksem.

```
int[] numbers = { 1, 2, 3, 4, 5 };  
foreach (int number in numbers)  
{  
    Console.WriteLine(number);  
}
```

Wykorzystanie wybranej biblioteki matematycznej

W języku C# istnieje wiele bibliotek matematycznych, które ułatwiają wykonywanie skomplikowanych obliczeń. Jedną z najczęściej używanych jest biblioteka System.Math, która oferuje szeroki zakres funkcji matematycznych, takich jak obliczenia trygonometryczne, logarytmiczne, potęgowe i wiele innych.

Najważniejsze funkcje biblioteki System.Math:

Funkcje trygonometryczne:

Math.Sin(double x): Zwraca sinus kąta x (w radianach).

Math.Cos(double x): Zwraca cosinus kąta x (w radianach).

Math.Tan(double x): Zwraca tangens kąta x (w radianach).

Funkcje potęgowe i pierwiastkowe:

Math.Pow(double x, double y): Zwraca wartość x podniesioną do potęgi y.

Math.Sqrt(double x): Zwraca pierwiastek kwadratowy z x.

Funkcje logarytmiczne:

Math.Log(double x): Zwraca logarytm naturalny z x.

Math.Log10(double x): Zwraca logarytm dziesiętny z x.

Funkcje zaokrąglające:

Math.Round(double x): Zaokrągla x do najbliższej liczby całkowitej.

Math.Ceiling(double x): Zaokrągla x w górę do najbliższej liczby całkowitej.

Math.Floor(double x): Zaokrągla x w dół do najbliższej liczby całkowitej.

Stałe matematyczne:

Math.PI: Stała π (pi), około 3.14159.

Math.E: Podstawa logarytmu naturalnego, około 2.71828.

Obliczanie wartości funkcji trygonometrycznych:

```
double angle = Math.PI / 4; // 45 stopni w radianach
```

```
double sinValue = Math.Sin(angle);
```

```
double cosValue = Math.Cos(angle);
```

```
double tanValue = Math.Tan(angle);
```

```
Console.WriteLine($"Sin(45°) = {sinValue}");
```

```
Console.WriteLine($"Cos(45°) = {cosValue}");
```

```
Console.WriteLine($"Tan(45°) = {tanValue}");
```

Obliczanie potęgi i pierwiastka:

```
double baseValue = 2;
```

```
double exponent = 3;
```

```
double power = Math.Pow(baseValue, exponent);
```

```
double sqrtValue = Math.Sqrt(power);
```

```
Console.WriteLine($"{baseValue} do potęgi {exponent} = {power}");
```

```
Console.WriteLine($"Pierwiastek kwadratowy z {power} = {sqrtValue}");
```

Obliczanie logarytmów:

```
double number = 100;
double logValue = Math.Log(number);
double log10Value = Math.Log10(number);
```

```
Console.WriteLine($"Logarytm naturalny z {number} = {logValue}");
Console.WriteLine($"Logarytm dziesiętny z {number} = {log10Value}");
```

Zaokrąglanie liczb:

```
double value = 5.67;
double roundedValue = Math.Round(value);
double ceilingValue = Math.Ceiling(value);
double floorValue = Math.Floor(value);
Console.WriteLine($"Zaokrąglone {value} = {roundedValue}");
Console.WriteLine($"Zaokrąglone w górę {value} = {ceilingValue}");
Console.WriteLine($"Zaokrąglone w dół {value} = {floorValue}");
```

Przykład 1: Obliczanie wartości funkcji trygonometrycznych

Program, który pobiera od użytkownika kąt w stopniach i oblicza wartości sinusa, cosinusa i tangensa tego kąta.

```
Console.Write("Podaj kąt w stopniach: ");
double degrees = double.Parse(Console.ReadLine());
double radians = degrees * (Math.PI / 180);
double sinValue = Math.Sin(radians);
double cosValue = Math.Cos(radians);
double tanValue = Math.Tan(radians);
Console.WriteLine($"Sin({degrees}°) = {sinValue}");
Console.WriteLine($"Cos({degrees}°) = {cosValue}");
Console.WriteLine($"Tan({degrees}°) = {tanValue}");
```

Przykład 2: Obliczanie potęgi i pierwiastka

Program, który pobiera od użytkownika dwie liczby: podstawę i wykładnik, a następnie oblicza wartość potęgi oraz pierwiastek kwadratowy z wyniku.

```
Console.Write("Podaj podstawę: ");
double baseValue = double.Parse(Console.ReadLine());
Console.Write("Podaj wykładnik: ");
double exponent = double.Parse(Console.ReadLine());
double power = Math.Pow(baseValue, exponent);
double sqrtValue = Math.Sqrt(power);
Console.WriteLine($"{baseValue} do potęgi {exponent} = {power}");
Console.WriteLine($"Pierwiastek kwadratowy z {power} = {sqrtValue}");
```

Program 3: Obliczanie logarytmów

Program, który pobiera od użytkownika liczbę i oblicza jej logarytm naturalny oraz dziesiętny.

```
Console.Write("Podaj liczbę: ");
double number = double.Parse(Console.ReadLine());
double logValue = Math.Log(number);
double log10Value = Math.Log10(number);
Console.WriteLine($"Logarytm naturalny z {number} = {logValue}");
Console.WriteLine($"Logarytm dziesiętny z {number} = {log10Value}");
```

Przykład 4: Zaokrąglanie liczb

Program, który pobiera od użytkownika liczbę zmiennoprzecinkową i zaokrągla ją do najbliższej liczby całkowitej, w górę oraz w dół.

```
Console.Write("Podaj liczbę: ");
```

```
double value = double.Parse(Console.ReadLine());
double roundedValue = Math.Round(value);
double ceilingValue = Math.Ceiling(value);
double floorValue = Math.Floor(value);
Console.WriteLine($"Zaokrąglone {value} = {roundedValue}");
Console.WriteLine($"Zaokrąglone w górę {value} = {ceilingValue}");
Console.WriteLine($"Zaokrąglone w dół {value} = {floorValue}");
```

Zadanie 5: Obliczanie odległości między punktami

Program, który pobiera od użytkownika współrzędne dwóch punktów w przestrzeni 2D i oblicza odległość między nimi.

```
Console.Write("Podaj współrzędne pierwszego punktu (x1 y1): ");
string[] point1 = Console.ReadLine().Split();
double x1 = double.Parse(point1[0]);
double y1 = double.Parse(point1[1]);
Console.Write("Podaj współrzędne drugiego punktu (x2 y2): ");
string[] point2 = Console.ReadLine().Split();
double x2 = double.Parse(point2[0]);
double y2 = double.Parse(point2[1]);
double distance = Math.Sqrt(Math.Pow(x2 - x1, 2) + Math.Pow(y2 - y1, 2));
Console.WriteLine($"Odległość między punktami wynosi: {distance}");
```

ZADANIA

Zadanie 1

Napisz program, który pobiera od użytkownika imię, nazwisko i wiek, a następnie wypisuje te informacje w jednym łańcuchu za pomocą interpolacji.

Zadanie 2

Napisz program, który pobiera od użytkownika imię, nazwisko i wiek, a następnie wypisuje te informacje w jednym łańcuchu za pomocą metody String.Format

Zadanie 3

Napisz program, który pobiera od użytkownika wynik z egzaminu (0-100) i wypisuje ocenę zgodnie z poniższą skalą:

- 90-100: 5
- 80-89: 4
- 70-79: 3
- 60-69: 2
- 0-59: 1

Zadanie 4

Napisz program, który pobiera od użytkownika liczbę i sprawdza czy jest ona parzysta czy nieparzysta.

Zadanie 5

Napisz program, który oblicza wartość wyrażenia M dla wybranych $a>0, b>0$

$$M = \frac{a+a}{b} + \frac{\frac{a}{b} * a}{b} + \frac{1}{a}$$

Zadanie 6

Napisz program, który oblicza wartości pierwiastków równania $x^2-2x-8=0$

Zadanie 7

Dla 3 ustalonych różnych wartości zmiennych a b c wyznacz liczbę największą

Zadanie 8

Wyznacz pierwiastki równania postaci $ax^2-bx-c=0$ dla zadanych parametrów a, b, c. Przeprowadzić dyskusję wyników w zależności od wartości współczynnika delta. Rozwiązanie podać tylko dla liczb rzeczywistych, w pozostałych przypadkach wyświetlić odpowiednie komunikaty.

Zadanie 9

Za pomocą pętli while, do-while lub for zrealizować następujące figury dla ustalonego parametru n.
np dla n=4

```
****  *      *      *
****  **     **     ***
****  ***    ***    *****
****  ****   ****   *****
```

Zadanie 10

Obliczyć pole powierzchni pola krzywej $y=x^2$ ograniczoną prostą $y=4$. Wynik podać dla iteracji odpowiednio z krokiem 0.5, 0.1, 0.01

Zadanie 11

Obliczyć pole powierzchni pola krzywej $y=x^2$ ograniczoną prostą $y=2x$. Wynik podać dla iteracji odpowiednio z krokiem 0.5, 0.1, 0.01